



AI-Integrated Full Stack: React, FastAPI with MySQL, and ML Model Deployment

— delivered by **Vi4 Academy** in joint venture with **LTTS**.

A hands-on 30-hour campus course that trains students to build production-grade full-stack applications and integrate ML models end-to-end, combining academic rigor with industry practices from Vi4 Academy and LTTS Academy.

Course Overview and Objectives

Overview

This intensive campus course blends frontend engineering (React), backend development (FastAPI + MySQL), and AI/ML model integration into a single, project-driven curriculum. Students learn modern tooling, deployment practices, and responsible AI considerations while delivering a working full-stack AI system.

Primary objectives

- **Teach practical skills** to design, implement, and deploy React frontends and FastAPI backends connected to MySQL.
 - **Enable ML integration** by training, serializing, and serving models through production APIs.
 - **Deliver industry readiness** through a mini project and capstone options that reflect real enterprise workflows.
 - **Foster teamwork and engineering discipline** with code quality, testing, containerization, and documentation.
-

Key Course Highlights

- **Duration:** 30 hours — ideal for semester labs or an intensive campus module.
- **Format:** Lecture + hands-on lab each session; team projects and demos.
- **Tech stack:** React, React Router, React Hook Form; FastAPI, Pydantic, SQLAlchemy, MySQL; scikit-learn / lightweight transformer options;
- **Assessment:** Weekly mini assignments, one industry-standard mini project, capstone (team of 4–5).

- **Industry mentorship:** Sessions and review checkpoints co-led by Vi4 Academy instructors and LTTS Academy industry mentors.
 - **Outcomes:** Deployable app, model artifact, project report, demo video.
 -  **GitHub Repository:** [Vi4Academy-Chennai/AI-Integrated-Full-Stack](#)
-

Learning Outcomes and Student Takeaways

By course end students will be able to:

- **Build responsive React applications** with routing, state management, form validation, and production deployment.
 - **Design and implement RESTful APIs** using FastAPI with secure authentication, database models, and background tasks.
 - **Integrate ML models** into backend services for real-time inference, including preprocessing pipelines and model versioning.
 - **Containerize and run** a full-stack system locally and prepare for cloud deployment.
 - **Apply engineering best practices:** testing, logging, monitoring, and basic model explainability.
 - **Work in teams** to plan, implement, and present an industry-grade AI system.
-

Delivery Model Logistics

Target audience

Undergraduate and postgraduate students with basic Python and JavaScript knowledge.

Prerequisites

- Introductory programming in Python and JavaScript.
- Basic HTML/CSS familiarity.
- Command line comfort.

Faculty and Mentorship

- **Vi4 Academy instructors** handle curriculum delivery, labs, and assessments.
- **LTTS Academy mentors** provide industry case reviews, guest lectures, and project evaluation.

Facilities and Resources

- Campus lab with internet access and high end processing capable machines.
- Optional GPU access for transformer experiments (if available).
- Repositories and starter templates provided by Vi4 Academy.

Assessment and Certification

- Graded assignments and project deliverables.
 - Certificate of completion co-branded by **Vi4 Academy** and **LTTS Academy** for students who meet passing criteria.
-

Course Outline (Hour-by-Hour, grouped by session)

Module 1: React UI (8 Hours)

- **Hour 1: React Fundamentals (Theory)**
 - Modern JavaScript refresher (HTML, CSS, ES6+ destructuring, arrow functions, modules).
 - Introduction to React, Virtual DOM, JSX syntax, and building components.
 - **Hour 2: React UI Lab (Practical)**
 - Set up a development environment using **Vite**.
 - Create your first functional components and render a simple static UI layout.
 - **Hour 3: State & Props (Theory)**
 - Passing data down via **Props**.
 - Managing internal component state using the `useState` hook.
 - **Hour 4: State & Props Lab (Practical)**
 - Build an interactive form with input fields for your future ML features (e.g., numeric sliders, text inputs).
 - Manage local state to capture and display user input instantly.
 - **Hour 5: Effects & Data Fetching (Theory)**
 - Handling side effects with `useEffect`.
 - Making HTTP requests using the native `fetch` API or `Axios`.
 - **Hour 6: Data Fetching Lab (Practical)**
 - Implement an asynchronous fetch call to a public mock API.
 - Render the fetched data dynamically in a list or card layout.
 - **Hour 7: Handling CORS & UI Polish (Theory)**
 - Understanding browser security policies and **CORS** (Cross-Origin Resource Sharing).
 - Styling components efficiently (Tailwind CSS or standard CSS modules).
 - **Hour 8: UI Polish Lab (Practical)**
 - Style your input form and prediction dashboard.
 - Prepare input validation states and loading spinners for asynchronous API requests.
-

Module 2: FastAPI & MySQL (8 Hours)

- **Hour 9: FastAPI Architecture & Pydantic (Theory)**
 - Introduction to modern Python web frameworks and asynchronous design.
 - Request payload validation using **Pydantic** models and automatic Swagger documentation.
 - **Hour 10: FastAPI Setup Lab (Practical)**
 - Initialize a virtual environment and install `fastapi` and `uvicorn`.
 - Create basic GET and POST endpoints with strict data validation schemas.
 - **Hour 11: Database Design & SQLAlchemy (Theory)**
 - Relational database fundamentals and schema design for MySQL.
 - Object-Relational Mapping (ORM) concepts using **SQLAlchemy 2.0**.
 - **Hour 12: Database Integration Lab (Practical)**
 - Connect FastAPI to a local MySQL instance.
 - Define SQLAlchemy models and configure session management dependencies.
 - **Hour 13: CRUD Operations & Routing (Theory)**
 - Structuring large applications using `APIRouter`.
 - Implementing secure Create, Read, Update, and Delete database operations.
 - **Hour 14: CRUD Lab (Practical)**
 - Build API endpoints to save user input submissions and past prediction logs into MySQL.
 - **Hour 15: Middleware & Security (Theory)**
 - Configuring CORS middleware in FastAPI to allow requests from your React frontend.
 - Managing environment variables (`.env`) for database credentials.
 - **Hour 16: Security & Middleware Lab (Practical)**
 - Finalize backend configuration, enable CORS, and test all routes via the interactive Swagger docs (`/docs`).
-

Module 3: Machine Learning Basics (8 Hours)

- **Hour 17: Supervised Learning Fundamentals (Theory)**
 - Overview of machine learning pipelines: data preprocessing, feature selection, and train/test splits.
 - Difference between **Regression** (predicting continuous values) and **Classification** (predicting categories).
- **Hour 18: Data Prep Lab (Practical)**
 - Load a clean dataset using `pandas`.

- Handle missing values and scale features using `scikit-learn`.
 - **Hour 19: Regression Models (Theory)**
 - Mechanics of Linear Regression.
 - Evaluation metrics: Mean Squared Error (MSE) and R^2 Score.
 - **Hour 20: Regression Lab (Practical)**
 - Train a Scikit-Learn Linear Regression model on sample data.
 - Evaluate performance metrics in a Jupyter notebook or standalone Python script.
 - **Hour 21: Classification Models (Theory)**
 - Mechanics of Logistic Regression and Decision Trees.
 - Evaluation metrics: Accuracy, Precision, Recall, and F1-Score.
 - **Hour 22: Classification Lab (Practical)**
 - Train a classification model and generate a confusion matrix to analyze performance.
 - **Hour 23: Model Serialization (Theory)**
 - Understanding how to persist trained models to disk using `joblib` or `pickle`.
 - Preparing models for production ingestion.
 - **Hour 24: Serialization Lab (Practical)**
 - Export your trained model into a `.pkl` file ready for backend integration.
-

Module 4: Capstone Mini Project (6 Hours)

- **Hour 25: Project Architecture & Model Loading**
 - Design the end-to-end workflow: React (Frontend) → FastAPI (Backend) → MySQL (Database) + Serialized ML Model.
 - Load the `.pkl` model directly into FastAPI startup events so it remains in memory for fast inference.
- **Hour 26: Backend Inference Endpoint Development**
 - Create a dedicated `/predict` POST endpoint in FastAPI that accepts incoming Pydantic validation schemas.
 - Pass incoming features directly into the loaded ML model and return the predicted result.
- **Hour 27: Database Logging Integration**
 - Update your prediction endpoint to save both the user input features and the resulting model prediction into your MySQL database for auditing/history.
- **Hour 28: Frontend Form & API Connection**
 - Build the final React form tailored to your chosen dataset features.

- Implement an `Axios` post request inside React to send user inputs to your FastAPI `/predict` endpoint.
 - **Hour 29: Full-Stack Integration & State Handling**
 - Capture the JSON response from FastAPI in your React component state.
 - Display the real-time prediction output gracefully on your UI dashboard with loading states and error handling.
 - **Hour 30: End-to-End Testing & Polish**
 - Run both servers concurrently (Vite for React, Uvicorn for FastAPI).
 - Perform end-to-end testing: submit live data through the React UI, check that FastAPI processes the ML prediction, saves the log to MySQL, and successfully returns the result back to the screen.
-

10 Capstone Real-Time Projects (Detailed)

1. House Price Predictor

- **Description:** An interactive real estate estimation app where users input property specifications to predict market values.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Linear Regression)
 - **Key Features:** Dynamic property feature form, real-time price calculation, and search history logging.
 - **Milestones:**
 1. Train and serialize the regression model.
 2. Build FastAPI `/predict` and database logging endpoints.
 3. Create the React UI form and link it via Axios.
 - **Learning Outcomes:** Connecting continuous regression models to REST APIs and persisting transaction logs in MySQL.
-

2. Customer Churn Predictor

- **Description:** A business intelligence tool to forecast whether a subscriber will cancel their service based on account usage.
- **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Logistic Regression)
- **Key Features:** Customer risk dashboard, probability score display, and audit trail of user queries.
- **Milestones:**

1. Train a binary classification model.
 2. Set up FastAPI backend with Pydantic validation.
 3. Build a React input interface with risk level badges.
- **Learning Outcomes:** Handling categorical user inputs and serving classification probability outputs via APIs.
-

3. Student Exam Score Predictor

- **Description:** An academic planning tool that estimates a student's final grade based on study habits and attendance.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Linear Regression)
 - **Key Features:** Input sliders for study hours/sleep, instant score projection, and student profile saving.
 - **Milestones:**
 1. Preprocess student dataset and train regressor.
 2. Implement backend CRUD routes for tracking.
 3. Design responsive student form in React.
 - **Learning Outcomes:** Managing numerical range inputs on the frontend and mapping them cleanly to regression features.
-

4. Diabetes Risk Classifier

- **Description:** A preliminary health screening tool that assesses clinical metrics to flag potential diabetes risk levels.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Logistic Regression)
 - **Key Features:** Medical metric input form, binary risk classification (**High** vs. **Low**), and secure query history.
 - **Milestones:**
 1. Train model on clinical indicators.
 2. Create secure FastAPI inference route.
 3. Build clean, accessible React health dashboard.
 - **Learning Outcomes:** Designing user-friendly medical data forms and storing structured evaluation logs.
-

5. Car Fuel Efficiency (MPG) Estimator

- **Description:** An automotive calculator that predicts miles per gallon based on engine specs and vehicle weight.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Linear Regression)
 - **Key Features:** Technical parameter inputs, MPG projection card, and vehicle search database archive.
 - **Milestones:**
 1. Train regression model on auto-mpg dataset.
 2. Build FastAPI endpoints for vehicle specifications.
 3. Develop React frontend dashboard.
 - **Learning Outcomes:** Translating multi-variable automotive metrics into real-time backend calculations.
-

6. Loan Approval Risk Analyzer

- **Description:** A financial decision-support tool that evaluates applicant credentials to recommend loan status.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Classification)
 - **Key Features:** Income/credit score input fields, automated approval decision (Approved / Rejected), and audit logging.
 - **Milestones:**
 1. Train classification model on financial datasets.
 2. Expose secure decision-making API endpoints.
 3. Build React form with status feedback UI.
 - **Learning Outcomes:** Implementing decision-support workflows with strict backend validation and logging.
-

7. Employee Salary Estimator

- **Description:** A career benchmarking tool that predicts expected annual compensation based on experience and skills.
- **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Linear Regression)
- **Key Features:** Experience and skill-rating inputs, estimated salary display, and market research query logging.
- **Milestones:**

1. Train salary prediction regressor.
 2. Set up FastAPI routing and database connection.
 3. Create interactive React career UI.
- **Learning Outcomes:** Handling multi-tier professional attributes and displaying formatted monetary outputs.
-

8. Email Spam Detector

- **Description:** A text-feature analysis tool that classifies message metrics as spam or legitimate.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Classification)
 - **Key Features:** Keyword/link frequency counters, classification confidence score, and flagged message history.
 - **Milestones:**
 1. Train text-feature classification model.
 2. Build backend inference and logging routes.
 3. Design React text-analysis dashboard.
 - **Learning Outcomes:** Processing derived text metrics and storing classification confidence metrics in MySQL.
-

9. Heart Disease Risk Screener

- **Description:** A cardiovascular health indicator app that estimates heart disease likelihood based on vital stats.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Logistic Regression)
 - **Key Features:** Blood pressure and cholesterol inputs, likelihood prediction badge, and patient screening logs.
 - **Milestones:**
 1. Train clinical risk classification model.
 2. Configure FastAPI backend and database models.
 3. Build intuitive React screening form.
 - **Learning Outcomes:** Building secure and structured health analytics features into a full-stack architecture.
-

10. Laptop Price Predictor

- **Description:** A hardware valuation tool that estimates secondary market pricing based on RAM, storage, and CPU specs.
 - **Tech Stack:** React (Vite), FastAPI, MySQL, Scikit-Learn (Linear Regression)
 - **Key Features:** Hardware drop-downs and sliders, estimated retail valuation, and search configuration archives.
 - **Milestones:**
 1. Train regression model on tech hardware specs.
 2. Implement FastAPI prediction endpoint.
 3. Create responsive React product configuration UI.
 - **Learning Outcomes:** Integrating complex hardware category states with backend regression pipelines.
-

In []: